

Experiment 1: RAW Q1 to Q22

Pure RLS is sometimes slower and not able to produce results as quickly as other alternate approaches. So, we are trying to develop an algorithm which can dynamically decide which enforcement approach is better for a given query. As of now we are considering four approaches mentioned in the compliance paper — Pure RLS, Indexed RLS, Secure View, and Query Rewrite. Through experiments we will try to identify the key factors that drive better performance for any particular approach.

Currently we are operating at the query level, but in further experiments we are aiming to develop a hybrid approach at the table level. However, for simplicity we are starting with query level analysis.

In Experiment 1, we execute the standard 22 TPC-H queries and record the planning time and execution time for each. This experiment serves as our baseline, giving us a glimpse of how each query performs without any policies or access control restrictions applied.

Query No	Planning Time (ms)	Exection Time (ms)
1	4.623	1359.72
2	4.118	77160.644
3	6.725	1010.615
4	4.211	1835.809
5	3.699	760.735
6	3.189	605.717
7	8.324	802.56
8	6.462	796.156
9	6.86	1316.33
10	4.773	929.387
11	5.2	292.894
12	5.537	800.53
13	1.849	2319.485

14	1.417	645.905
15	0.952	617.666
16	2.11	310.632
17	0	0
18	4.201	4443.423
19	6.57	907.677
20	0	0
21	0	0
22	10.304	227.401

Here 0 means timeout.

Observations after the results:

⇒ Q2 is slow because of the **correlated subquery** that recalculates `min(ps_supplycost)` for every matching part row, joining 4 tables (`partsupp`, `supplier`, `nation`, `region`) repeatedly. The inner subquery essentially re-executes a 4-table join **thousands of times** — once per candidate part — making it extremely expensive despite the small final result of 100 rows.

```

SELECT
    s_acctbal,
    s_name,
    n_name,
    p_partkey,
    p_mfgr,
    s_address,
    s_phone,
    s_comment
FROM
    part,
    supplier,
    partsupp,
    nation,
    region
WHERE

```

```

p_partkey = ps_partkey
AND s_suppkey = ps_suppkey
AND p_size = 15
AND p_type LIKE '%BRASS'
AND s_nationkey = n_nationkey
AND n_regionkey = r_regionkey
AND r_name = 'EUROPE'
AND ps_supplycost = (
    SELECT MIN(ps_supplycost)
    FROM
        partsupp,
        supplier,
        nation,
        region
    WHERE
        p_partkey = ps_partkey
        AND s_suppkey = ps_suppkey
        AND s_nationkey = n_nationkey
        AND n_regionkey = r_regionkey
        AND r_name = 'EUROPE'
)
ORDER BY
    s_acctbal DESC,
    n_name,
    s_name,
    p_partkey
LIMIT 100;

```

⇒ Q17 has a **correlated subquery** that runs once for every single row in **lineitem** (6 million rows) — each time recalculating `avg(l_quantity)` by scanning **lineitem** again. So it's essentially doing **6 million × lineitem scans** which is practically infinite work. No index can help because the correlation ties it to the outer query's current row.

```

SELECT
    SUM(l_extendedprice) / 7.0 AS avg_yearly

```

```

FROM
    lineitem,
    part
WHERE
    p_partkey = l_partkey
    AND p_brand = 'Brand#53'
    AND p_container = 'MED BAG'
    AND l_quantity < (
        SELECT
            0.7 * AVG(l_quantity)
        FROM
            lineitem
        WHERE
            l_partkey = p_partkey
    );

```

⇒ Q20 has **triple nested subqueries** where the innermost correlated subquery recalculates `sum(l_quantity)` on lineitem **for every partsupp row** that matches. Combined with `like '%ivory%'` forcing a full table scan on part, and then filtering through partsupp → lineitem repeatedly, it's essentially **multiple full lineitem scans nested inside each other**.

```

SELECT
    s_name,
    s_address
FROM
    supplier,
    nation
WHERE
    s_suppkey IN (
        SELECT
            ps_suppkey
        FROM
            partsupp
        WHERE
            ps_partkey IN (

```

```

SELECT
    p_partkey
FROM
    part
WHERE
    p_name LIKE '%ivory%'
)
AND ps_availqty > (
    SELECT
        0.5 * SUM(l_quantity)
    FROM
        lineitem
    WHERE
        l_partkey = ps_partkey
        AND l_suppkey = ps_suppkey
        AND l_shipdate >= DATE '1995-01-01'
        AND l_shipdate < DATE '1995-01-01' + INTE
RVAL '1' year
)
)
AND s_nationkey = n_nationkey
AND n_name = 'FRANCE'
ORDER BY
    s_name;

```

⇒ Q21 has **two correlated subqueries (EXISTS and NOT EXISTS) both scanning lineitem** for every row in the outer lineitem join. So lineitem is joined 3 times total (I1, I2, I3) — the EXISTS checks one full lineitem scan and NOT EXISTS checks another, **repeated for every row in the already-expensive 4-table join** of supplier × lineitem × orders × nation.

```

SELECT
    s_name,
    COUNT(*) AS numwait
FROM
    supplier,

```

```

lineitem l1,
orders,
nation
WHERE
s_suppkey = l1.l_suppkey
AND o_orderkey = l1.l_orderkey
AND o_orderstatus = 'F'
AND l1.l_receiptdate > l1.l_commitdate
AND EXISTS (
    SELECT *
    FROM lineitem l2
    WHERE
        l2.l_orderkey = l1.l_orderkey
        AND l2.l_suppkey <> l1.l_suppkey
)
AND NOT EXISTS (
    SELECT *
    FROM lineitem l3
    WHERE
        l3.l_orderkey = l1.l_orderkey
        AND l3.l_suppkey <> l1.l_suppkey
        AND l3.l_receiptdate > l3.l_commitdate
)
AND s_nationkey = n_nationkey
AND n_name = 'ARGENTINA'
GROUP BY
s_name
ORDER BY
numwait DESC,
s_name;

```

⇒ Now in the next experiment, we will apply a single policy on each of the 4 tables and evaluate how all 4 enforcement approaches perform against the same 22 queries.